ME5413 Report

Homework 2: Localisation

MAI Jingyang

National University of Singapore

maij@u.nus.edu

A0304610X

College of Design and Engineering

Department of Mechanical Engineering

National University of Singapore

Session 2024/2025

I. Introduction

The classic SLAM framework is composed of 4 parts, Sensor data acquisition, Frontend, Map Building and Loop Closure & Backend. The first part mainly refers to the utilizing of 2D/3D LiDAR and camera sensors, as well as wheel encoders and IMU sensors in the domain of mobile robot. The second part focus on estimate sensor's ego-motions between adjacent frames. It provides an initial estimate of the sensor's pose, and/or the trajectory with possible small errors that would be used to construct a map in the third part. Finally in the fourth part optimization is applied to generate the fully optimized trajectory and map.

The first task of this assignment is to implement Iterative Closest Point (ICP), an algorithm typically used in Frontend part of SLAM, by processing 2 set of point clouds, typically obtained by 2D/3D LiDAR, to find the optimal rigid transformation (rotation and translation) that minimizes the distance between corresponding points in the two point clouds. The second task would be to run and evaluate popular an open-sourced SLAM algorithm in ROS. The title of each task is shown as follows,

- Task1: Point Cloud Registration,
  - Task1.1: ICP with known correspondence;
  - Task1.2: ICP with unknown correspondence.
- Task2: Running SLAM with Google Cartographer[1].

In task1.1, we implemented ICP based on Singular Value Decomposition (SVD), and successfully obtained the transformation matrix between two point clouds, achieving a mean error of 0.015891 w.r.t. all corresponding point pairs. In task1.2, we develop an improved ICP including rough registration, outlier point pair rejection, and finally achieve a mean error of 0.015895 in 133 iterations and approximate 30 seconds, which shows high robustness and powerful convergence capability. In task2, we are required to use a simulated mobile robot, Jackal(with an IMU, a 2D LiDAR, and a 3D LiDAR), to perform SLAM based on Google Cartographer. In this task, 2D LiDAR is used to perform 2D SLAM, finally achieving a relatively high-resolution map with an absolute RSME of 0.101842.

II. Methodology

A. SVD-Based ICP

With correspondence, reference point cloud 1 and point cloud 2 to be registered can be easily registered by ${}^1T_2$, which is calculated as follows,

$$ {}^1T_2 = \begin{bmatrix} {}^1R_2 & {}^1\boldsymbol{t}_2 \\ \mathbf{0}^T & 1 \end{bmatrix}, $$

$$ {}^1R_2 = VU^T, $$

$$ {}^1\boldsymbol{t}_2 = \boldsymbol{u}_1 - {}^1R_2\boldsymbol{u}_2, $$

where $\mathbf{0} = [0 \quad 0 \quad 0]^T$, matrix V and U are obtained by performing SVD on H ($U\Sigma V^T = H$, where $H = \sum_{i=1}^{N_2} \boldsymbol{p}_{2i}{}'\boldsymbol{p}'^T_{1i}$, $\boldsymbol{p}'_{2i}$ and $\boldsymbol{p}'_{1i}$ correspond to the centered i-th point of two point clouds respectively), $\boldsymbol{u}_1 = \frac{1}{N_1}\sum_{i=1}^{N_1} \boldsymbol{p}_{1i}$ and $\boldsymbol{u}_2 = \frac{1}{N_2}\sum_{i=1}^{N_2} \boldsymbol{p}_{2i}$. Ideally, the number of points in point cloud 1 N_1 is equal to N_2.

B. Improved ICP (for Task1.2)

Our improved ICP has 3 key features inspired by [2]. (1) We introduced KD-Tree to match the closest point pairs, i.e., the correspondence between points. (2) We performed a rough registration between two point clouds before iterative updates, where we first calculate the 3D center point of each point cloud and the translation transformation between the two center points, which we will use for transferring each point of the point cloud to be registered (point cloud 2). (3) Outlier point pairs outside a set threshold range would be rejected during each iteration of ICP. The whole improved ICP framework is shown as below.

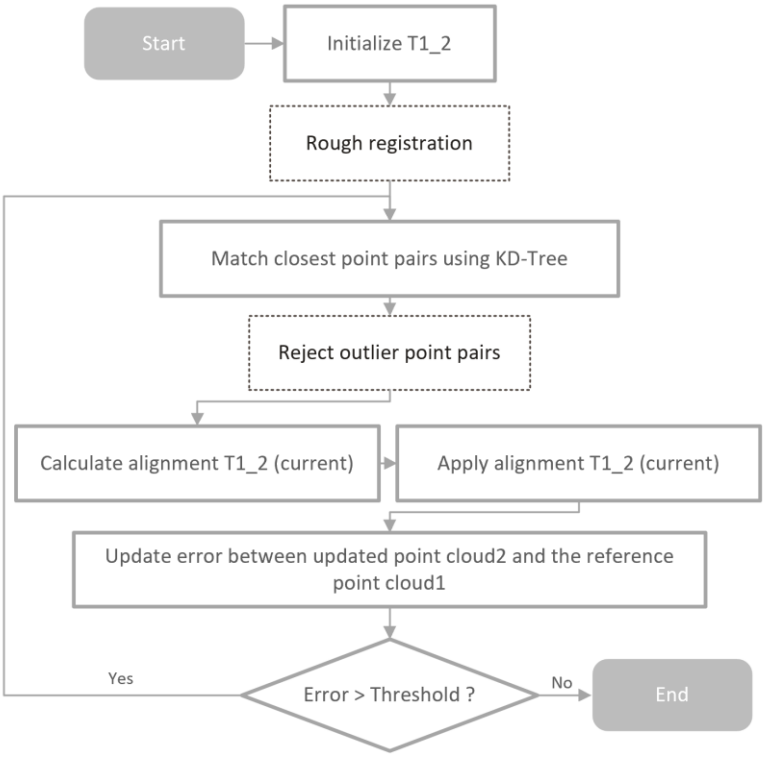


Figure 1 Improved ICP framework.

C. *Cartographer Parameter Modifications/Tunings and Parallelization*

Cartographer starts SLAM by launching a launch file, in which a lua configuration file is included, required to align to the customer mobile robot configuration and scenario. The core mission in task2 is to modify and fine-tune parameters in this lua configuration file. During task2, (1)First, we read sensor data and parameters for better modifying the initial configuration. (2)Second, we develop a parallelization method to adjust parameters concurrently, enabling intuitive comparison of the impact of different values of the same parameter on the mapping effect. The parameter modifications/tunings of lua file that achieves the best balance between evo RMSE and mapping effect are shown in the Table 1 below, and the parallelization effect is attached to the appendix.

Table 1 Parameter Modifications/Tunings in Customer lua File.

	Name	Default	Modified	↑/↓
Local SLAM (TRAJECTORY_BUILDER_2D)	submaps.num_range_data	90	6000	↑↑
	min_range	0.	0.3	↑
	max_range	10.	6.	↓
	voxel_filter_size	0.025	0.3	↑
	num_accumulated_range_data	1	2	↑
	use_imu_data	true	true	-
	use_online_correlative_scan_matching	false	true	↑
	ceres_scan_matcher.occupied_space_weight	1.	1.	-
	ceres_scan_matcher.translation_weight	10.	10.	-
	ceres_scan_matcher.rotation_weight	40.	50.	↑
Global SLAM (POSE_GRAPH)	optimize_every_n_nodes	90	20	↓
	constraint_builder.max_constraint_distance	15.	5.	↓
	optimization_problem.huber_scale	1e1	1e2	↑
	constraint_builder.min_score	0.55	0.65	↑

III. Experiments and Results

A. *Task1.1: ICP with known correspondence*

We obtain a mean error of 0.015891 w.r.t. all corresponding point pairs, and the registration effect with the homogeneous transformation matrix are shown as below.

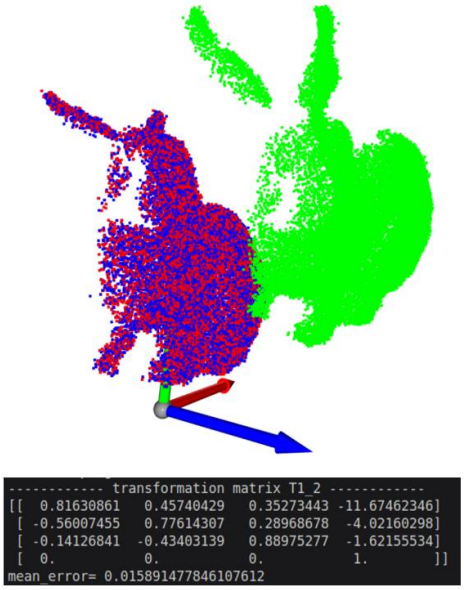
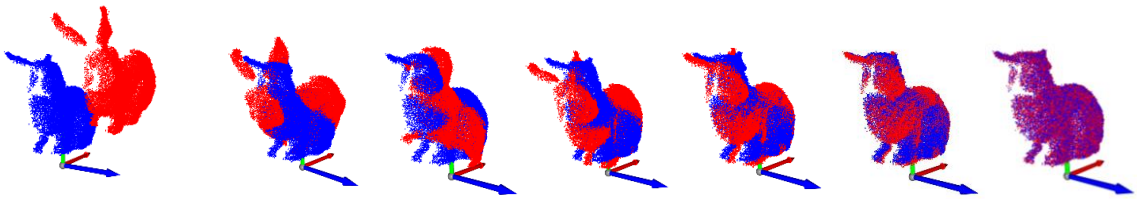


Figure 2 Registration Effect with Homogeneous Transformation Matrix of Task1.1

B. *Task1.2: ICP with unknown correspondence*

The following figures and table show effects of initial point cloud state, rough registration, the 5th, 10th, 50th, 90th, 133rd (final) ICP iteration, as well as the mean error and the time cost. The final column in the table show the comparison between the final converged effect of improved ICP and the effect of task1.1, which appear ultimately close. The Figure 4 shows the convergence curve of mean error along the iteration process.



	ICP with unknown correspondence (Task1.2)							Task1.1
Iteration	0	Rough Regis.	5	10	50	90	133	
mean error		2.601946	2.195185	1.936563	0.837117	0.341279	0.015895	0.015891
time cost			1.13s	2.24s	10.83s	19.93s	29.49s	

Figure 3 The Improved ICP effects of initial point cloud state, rough registration, the 5th, 10th, 50th, 90th, 133rd iteration.

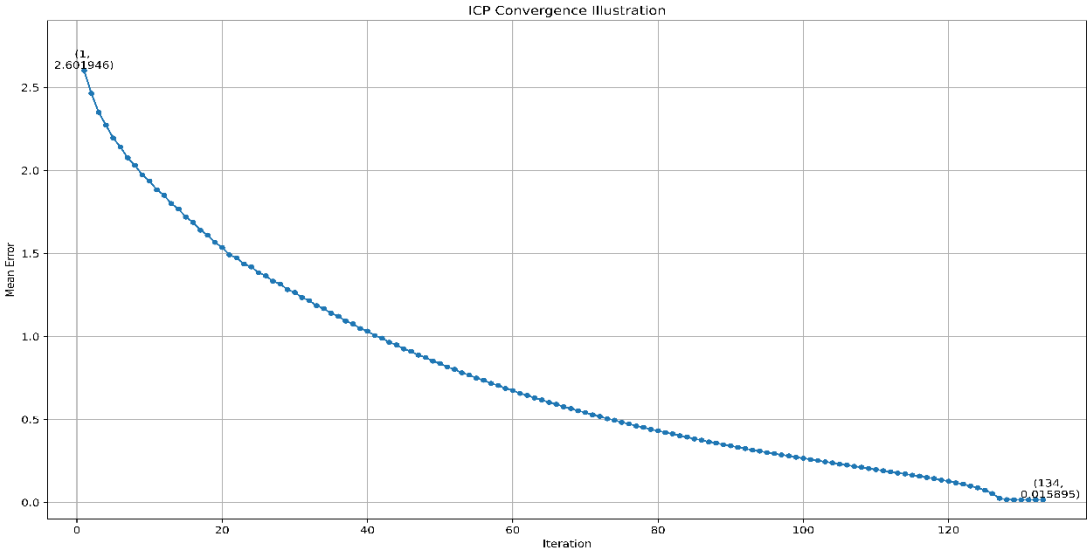


Figure 4 Improved ICP Convergence Illustration.

The final accumulative transformation matrix is shown as below, which achieves an ultimately close result compared to the one obtained in Task 1.1.

```
accumulated T =
[[  0.8162684    0.45745193   0.35276569 -11.67572779]
 [ -0.56013559   0.77609608   0.28969464  -4.02102384]
 [ -0.1412587   -0.4340652    0.88973782  -1.62095979]
 [  0.           0.           0.           1.        ]]
mean_error= 0.015895022230802813
------- fully converged! -------
```

Figure 5 Final Homogeneous Transformation Matrix of Task1.2

C. *Task2: Running SLAM with Google Cartographer*

First, we read the scanning range of the 2D LiDAR by subscribing to the /scan topic for better modifying the initial configuration, the range of 2D LiDAR is 271. Then we perform parallelization parameter tuning to achieve the best balance between evo RMSE and mapping effect.

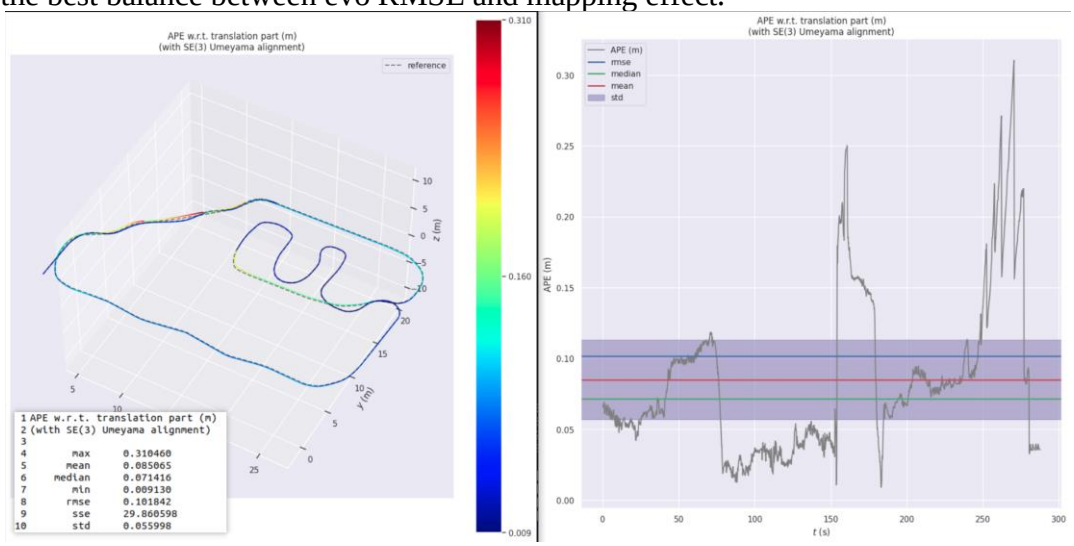


Figure 6 SLAM Trajectory and Evo Evaluation.

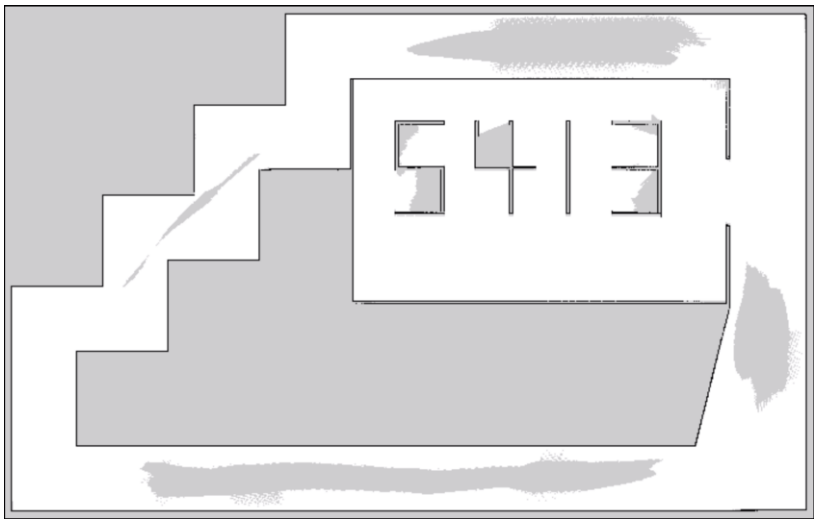


Figure 7 Best Mapping Effect.

IV. Discussion

In task1.1, the ICP algorithm with correspondence shows very advanced closed-form solution computational efficiency. In task1.2, we used KD-Tree and ICP algorithm to successfully register two point cloud achieving ultimately close effect as task1.1, which verifies the powerful convergence capability of ICP algorithm.

In task2, we used a parallel approach to adjust parameters and achieved extremely low evo RSME and relatively accurate mapping. However, there are still inaccuracies in the map, such as shadows and blurred corners, which still require further parameter fine-tuning. We also unexpectedly noticed that the evo RMSE and the accuracy of the map do not conform to a very strict linear relationship. For details, please see the Appendix.

V. Conclusion

In this assignment we successfully implement several SLAM algorithms as well as incorporate them with realistic data and ROS, visualize the results and evaluate them with comparison.

Acknowledgements

The author would like to express gratitude to Bai Yue, for her extremely effective sh command file to facilitate the testing speed of task2. The author also would like to thank Anurag Roy, by whom the illustration figures in task1 are inspired so much.

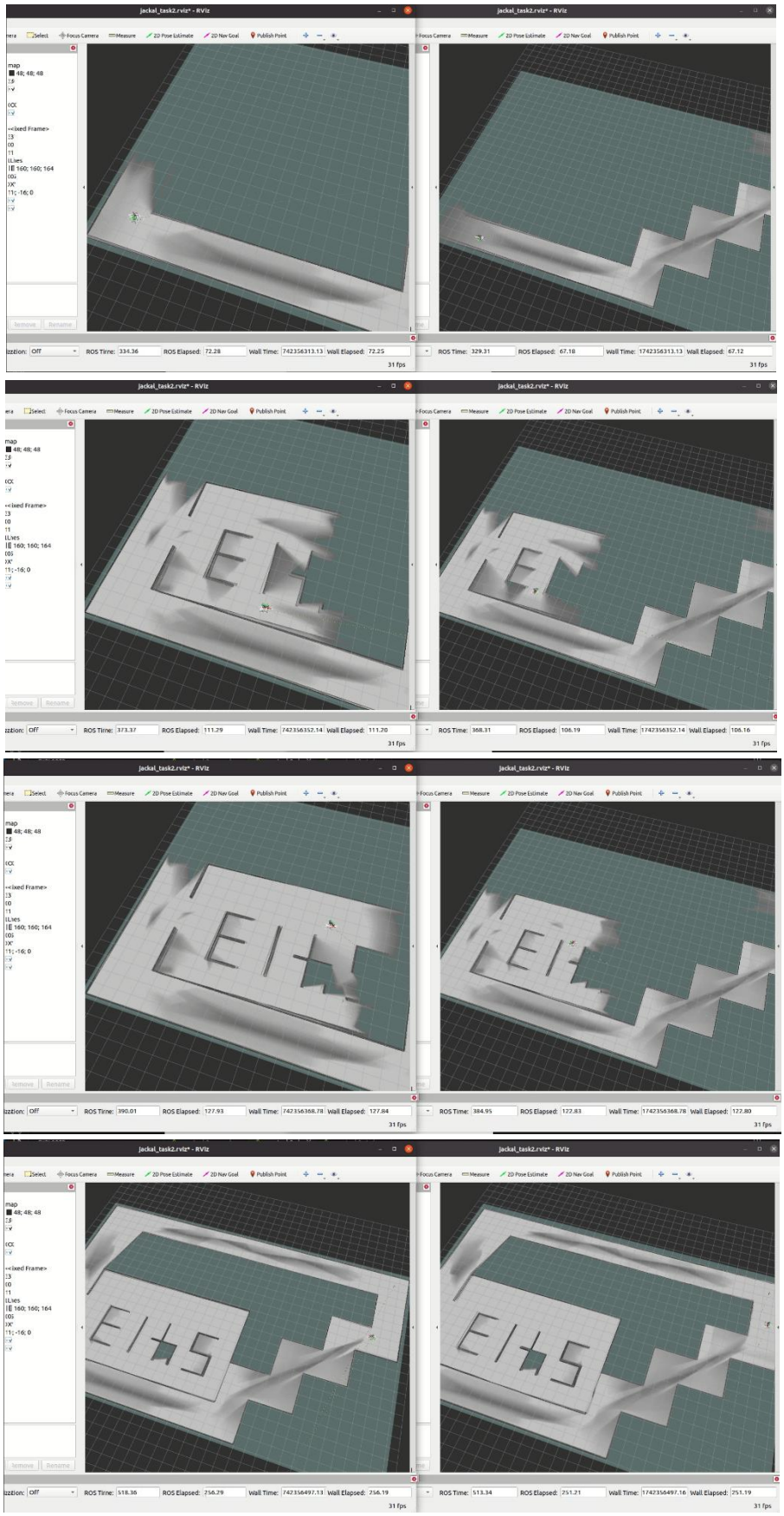
Reference

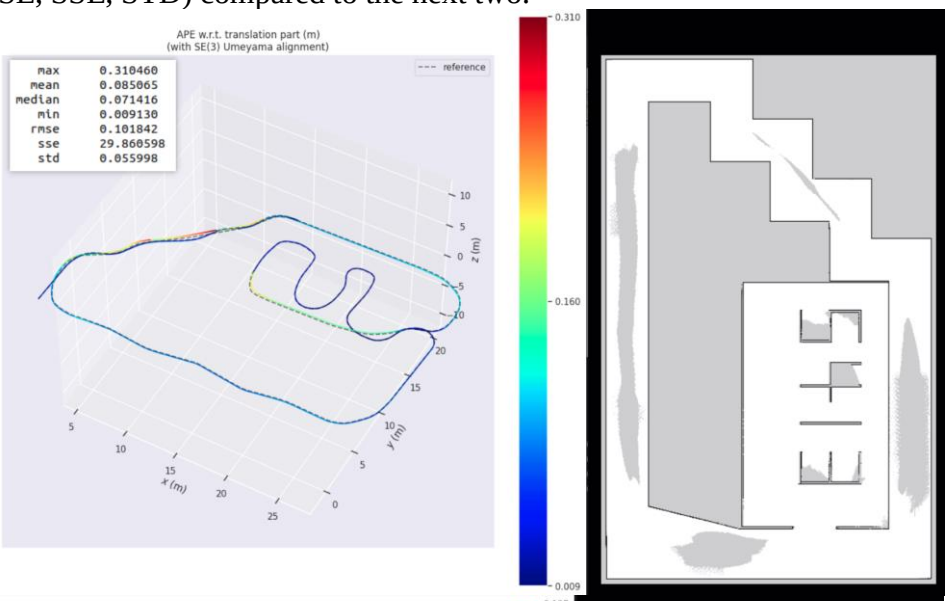
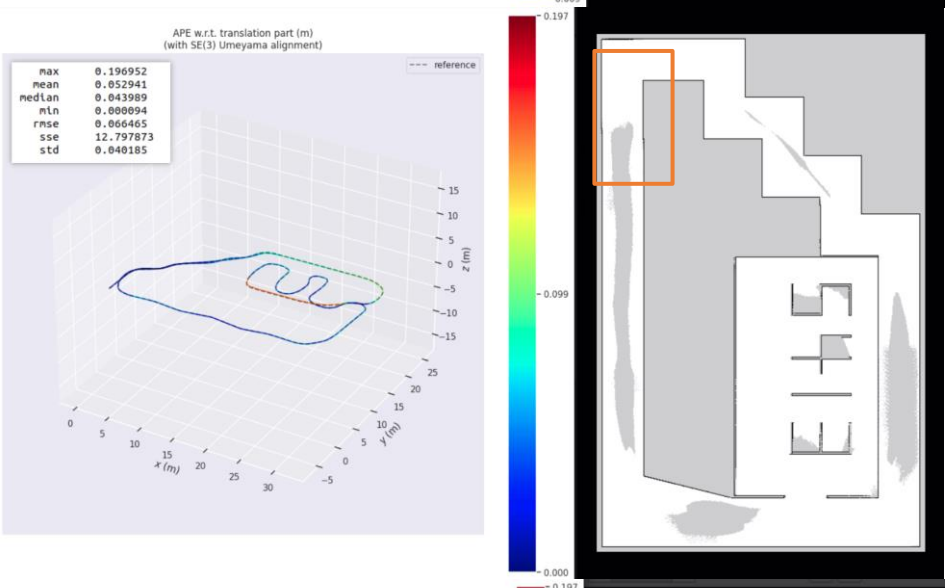
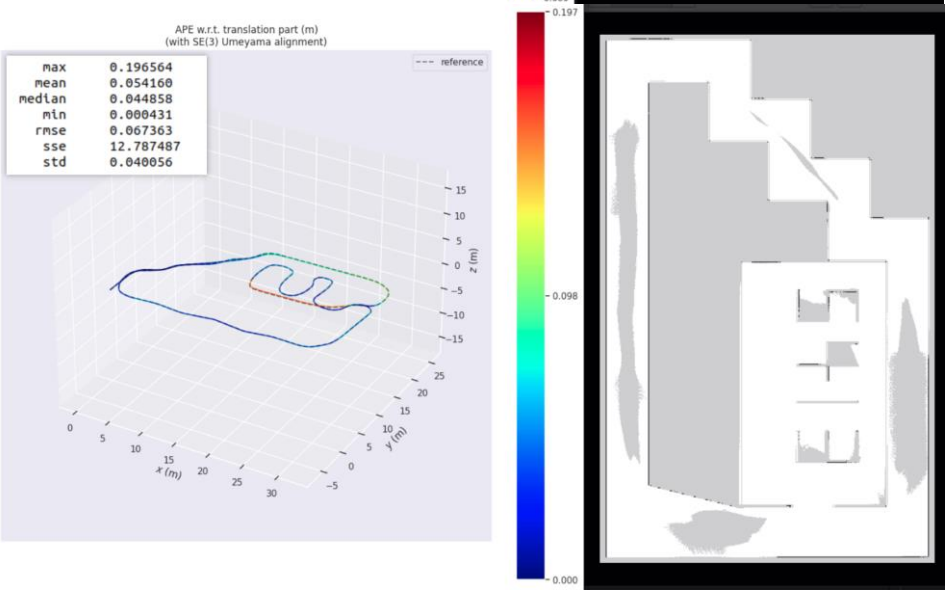
[1] "Cartographer ROS Integration — Cartographer ROS documentation." Accessed: Mar. 20, 2025. [Online]. Available: https://google-cartographer-ros.readthedocs.io/en/latest/index.html

[2] S. Rusinkiewicz, "Lecture 10: Acquisition and registration," *Princeton University*, Fall 2018. [Online]. Available: https://www.cs.princeton.edu/courses/archive/fall18/cos526/notes/cos526_f18_lecture10_acquisition_registration.pdf. [Accessed: Mar. 20, 2025].

A Parallelization Parameter Tuning

In task2, we use different roscore ports to run several independent ROS environments to achieve parallel parameter adjustment. The effects are shown as below.



B Comparison between SLAM Rrajectory Ouput and Generated Map Effect

Three comparisons are shown as below. The first one shows the best mapping effect, but with higher evo scores (RMSE, SSE, STD) compared to the next two.